CascadeNet — Speaker Notes

NSDI '26 | 15-minute talk + 5-minute Q&A

Slide 1 | Title *(~30s)*

Good afternoon everyone. My name is Xinyu Li. I'm from Carnegie Mellon University. Today I'm going to present CascadeNet, an end-to-end framework for generating network traffic with high-fidelity temporal patterns. This is joint work with collaborators from NYU Shanghai, Peking University, USTC, and HKUST.

Slide 2 | Why Synthetic Network Traces? *(~1 min)*

First. let me start with the motivation. As we all know, real network traces are essential for research such as traffic classification, anomaly detection, behavior analysis, and many other tasks. But getting access to real traces is hard. Privacy concerns and proprietary restrictions make it difficult to share real-world data.

This has driven a lot of interest in synthetic trace generation. The existing approaches are simulation, model-based, and machine-learning-based. Among all, machine learning methods are the most promising because they learn patterns directly from data without heavy domain expertise.

However, there's a key gap. Existing ML-based generators fail to capture temporal patterns, that is, how network activity evolves over time. This includes things like packet rate, throughput and burstiness. This is a problem because many downstream tasks depends on these properties.

Slide 3 | Existing Generators Distort Temporal Patterns *(~1.5 min)*

To make this concrete, here's the packet rate of the CAIDA dataset, and synthetic traces learnt from CAIDA by different ML-based models. The gray line is the real trace.

For models like E-WGAN-GP and STAN, they completely distort the packet rate pattern. For NetShare, which is the state-of-the-art, it actually learns a nonexistent periodic pattern, [point to figure] these periodic spikes that don't exist in the real data.

Now for our model, CascadeNet. the red dashed line. It closely tracks the pattern of the real packet rate.

So now the question is: why do existing generators struggle? We did a thorough analysis, and find that the root cause lies in the data representation

Slide 4 | Data Representation — The Root Cause *(~2 min)*

Let me walk through the three existing approaches.

Approach 1 treats the trace as a collection of independent records. Methods like E-WGAN-GP follow this. Since it completely ignores the order and relationships between records, there's no way to capture temporal patterns.

Approach 2 treats the trace as a sequence of records, like STAN does. This is better in principle, but when you have tens of thousands of packets, it's extremely hard for models to learn long-term dependencies from such long sequences.

Approach 3, used by NetShare, splits the trace into flows and generates each flow as a record sequence. This reduces sequence length, but elephant flows can still easily have thousands of records. So models still struggle with temporal dynamics.

Our key insight is this: temporal patterns like packet rate and throughput are *aggregated features*. They emerge from combining information across many records within a time interval. So instead of asking models to infer these patterns from raw records, we should let models explicitly learn them as time series.

That's our fourth approach — we split the trace into flows as approach 3 did, and aggregate each flow into a fixed-length time series of packet count within each time interval. This gives us two advantages: first, higher fidelity, because the model directly learns the temporal patterns, and second, better scalability, because the length of time series is fixed no matter how many packets are in the flow.

Slide 5 | CascadeNet Workflow *(~1 min)*

Here's the overall CascadeNet workflow. It has three stages: [points to figure] data preprocessing, training & generation, and data postprocessing.

In data preprocessing, we split the trace into flows by 5-tuple, encode the IP addresses and ports, aggregate each flow into a time series capturing packet rate and throughput, and encode the remaining record fields.

The core is the training and generation stage, where CascadeGAN — our hierarchical generative model — learns to generate all three components: IP 5-tuples, aggregated time series, and record fields.

In post-processing, we decode the model outputs back to readable fields. Specifically, we generate timestamps from the aggregated time series to produce the actual synthetic trace.

Slide 6 | CascadeGAN Architecture *(~1.5 min)*

Now let me explain CascadeGAN, the hierarchical generative model, and the core of our framework.

The challenge of the training & generation phase is that our three components — 5-tuples, time series, and record fields — are fundamentally different in nature. 5-tuples are fixed-dimensional vectors, time series are variable-length sequences, and record fields vary in count per time step. Using a single model for all three is impractical, but using three independent models would break their interdependencies.

Our solution is CascadeGAN, a hierarchical model with three pairs of generator and discriminator. It works from coarse to fine.

Generator 1 is an MLP that generates IP 5-tuples from noise. Generator 2 is an LSTM that takes the generated 5-tuples as input and produces the aggregated time series. This captures flow-level temporal dynamics. Generator 3 is an MLP that takes both the 5-tuples and a sliding window of the time series to generate record fields.

The key idea is that we convert the hard problem of learning a single joint distribution over different components into learning multiple conditional distributions — which is much more tractable.

For training, we use a two-phase strategy: first pretrain each GAN pair separately with real inputs, then finetune all of them jointly. This avoids what we call “cascading divergence” — where errors in Generator 1 would mislead Generator 2 and so on.

Slide 7 | Timestamp Generation & Optimizations *(~1 min)*

Two more technical pieces. First, since CascadeNet learns temporal patterns from aggregated time series rather than the actual timestamps, we need to recover timestamps in post-processing. We developed three techniques — equidistant, sub-period, and ML-based. The ML-based approach works best and is our default. In this approach, the third GAN additionally learns relative timestamps within each time step,

We also address two data distribution challenges. First, the aggregated time series are very sparse — up to 88% zeros on some datasets. To solve this, we use a zero-inflation method that adds a binary feature indicating whether each time step is active. Second, network traces have extreme class imbalance between mouse flows and elephant flows. To solve this, we use flow-level conditional inputs — like flow duration and max packet rate — to prevent mode collapse.

Both techniques are validated in our ablation studies and significantly improve fidelity.

Slide 8 | Evaluation Setup *(~45s)*

For evaluation, we use four diverse IPv4 header traces: CAIDA from a backbone ISP, DC from a university data center, CA from a cyber defense competition, and TON_IoT from IoT sensors. Thay vary significantly in scale, duration, and flow characteristics.

We compare against six baselines spanning different generation paradigms — tabular GANs, flow GANs, auto-regressive models, diffusion models, and transformer-based methods.

We evaluate along four dimensions: temporal fidelity, statistical fidelity, downstream task performance, and diversity plus scalability.

Slide 9 | Temporal Fidelity Results *(~1 min)*

Here are the results of temporal fidelity. We evaluate six temporal metrics: interarrival time, flow-level interarrival time, flow duration, average throughput, burstiness, and long-term dependency. We measure the distance between real distribution and synthetic distribution extracted from the synthetic trace using Earth Mover's Distance, or EMD for short. Lower is better.

On both CAIDA and DC, CascadeNet — the red hatched bars — most of the time has the lowest EMD across all metrics. Overall, CascadeNet achieves 41% to 76% improvement over baselines.

You might notice that REaLTabFormer, the pink dotted bar, also shows decent temporal fidelity, but as we'll see later, that comes at the cost of data diversity: RealTabFormer achives high fidelity by memorizing a large fraction of the training data.

Slide 10 | Statistical Fidelity *(~30s)*

For statistical fidelity, such as source/destination address distributions, port distributions, packet lengths, CascadeNet performs comparably to baselines. This shows that our focus on temporal patterns doesn't come at the cost of statistical accuracy. We maintain competitive performance on all nine statistical features.

Slide 11 | Burst Analysis & Throughput Prediction *(~1 min)*

Now the downstream tasks, which is where temporal fidelity really matters.

On the left is burst analysis. We follow a recent IMC methodology to compare bursts in real vs. synthetic traces. CascadeNet achieves the best or near-best EMD across all five burst metrics: burst duration, time between bursts, network utilization, and packet sizes during and outside bursts.

On the right is throughput prediction. We train six different ML models on synthetic throughput data and measure how well they predict real throughput. CascadeNet produces the lowest normalized MAE across most models. This means that predictors generalize well to real traffic when they are trained on synthetic data generated by CascadeNet.

These results confirm that high temporal fidelity directly translates to more reliable downstream analyses.

Slide 12 | Anomaly Detection *(~45s)*

For anomaly detection, we use the NetML library with one-class SVM. We train detectors on synthetic traces and evaluate their F1 score on detecting anomalies in real traces.

CascadeNet maintains higher F1 than NetShare across most flow representations — interarrival time, packet size, combined features, and statistics. These results are consistent across six different anomaly detection models that we report in the appendix.

Slide 13 | Diversity & Scalability *(~1 min)*

We also evaluates the diversity and scalability of different models. First, for diversity: 99.9% or more of CascadeNet's generated records are different from the training data. This contrasts sharply with REaLTabFormer, which memorizes between 1.7% and 56.5% of training records — which explains why its fidelity numbers looked good earlier.

Second, for scalability, these plots show training time and generation time on a log scale for different models. Compared to the state-of-the-art model NetShare, CascadeNet is 7 to 25 times faster in training, and 9 to 62 times faster in generation.

The reason for this large speedup is that, our aggregated time series have fixed, short lengths, at most 200 steps no matter how many packets are in a flow. In comparison. NetShare has to process elephant flows with tens of thousands of records as sequences, which is inherently much slower.

Slide 14 | Conclusion *(~30s)*

To summarize: we propose a novel data representation that aggregates flows into time series to explicitly expose temporal patterns. We design CascadeGAN, a hierarchical model that learns IP 5-tuples, time series, and record fields from coarse to fine. CascadeNet achieves 41 to 76 percent better temporal fidelity, comparable statistical fidelity, and superior downstream performance — all while being 7 to 25 times faster to train than the state of the art.

The code is publicly available on GitHub. Thank you for your attention, and I'm happy to take questions.

Timing Summary

Target: ~14 minutes (1 min buffer for transitions)

Slide	Topic	Time	Cumul.	Notes
1	Title	0:30	0:30	
2	Motivation	1:00	1:30	
3	Problem (packet rate)	1:30	3:00	Key figure
4	Root cause (representations)	2:00	5:00	Core argument
5	Workflow	1:00	6:00	
6	CascadeGAN + Training	1:30	7:30	Technical core
7	Timestamps & optimizations	1:00	8:30	
8	Eval setup	0:45	9:15	Quick
9	Temporal fidelity	1:00	10:15	Main result
10	Statistical fidelity	0:30	10:45	Brief
11	Burst + throughput	1:00	11:45	
12	Anomaly detection	0:45	12:30	
13	Diversity & scalability	1:00	13:30	
14	Conclusion	0:30	14:00	

Anticipated Q&A

Q: Why GANs instead of diffusion models or transformers?

We evaluated both. Diffusion models like NetDiffusion require weeks to generate traces for large datasets due to their flow-to-image conversion. Transformer-based models like REaLTabFormer tend to memorize training data rather than truly generalize — up to 56.5% copied records. GANs offer the best balance of quality, diversity, and efficiency for this task.

Q: How do you choose the time series length?

We recommend ≤200. Appendix H shows that lengths ≥500 significantly degrade fitting capability. Even length 20 gives adequate fidelity thanks to our timestamp recovery technique. The sweet spot depends on dataset duration — shorter traces can use shorter series.

Q: What about differential privacy?

We found DP training causes approximately 100x slowdown in PyTorch with significant fidelity loss. Previous work by NetShare also showed that even weak DP severely compromises fidelity. IP anonymization techniques are orthogonal and can be applied separately. We leave privacy-preserving generation as future work.

Q: Can CascadeNet handle payload data?

Not currently — we focus on IPv4 headers. Adding a payload generation layer to the hierarchy is a promising direction, but payloads contain application-layer protocols and natural language content, making this substantially more complex.

Q: How does CascadeNet handle inter-flow dependencies?

CascadeNet generates flows independently. For tasks like congestion control evaluation that need inter-flow interactions, you'd need a flow dependency graph. While domain experts could construct such graphs manually, a more promising direction is using ML to automatically infer these relationships from traces.

Q: What about the conditional input — isn't that leaking information?

The conditional inputs (flow duration, max packet rate, etc.) are flow-level statistics used to prevent mode collapse. During generation, they're sampled from the learned distribution of the training data. Users can also manually adjust these proportions to customize the generated trace for specific use cases.